

Proof Buddy — Data Flow

This document traces how data moves through the Proof Buddy system at each stage of a user's interaction. It is intended to give an AI or developer a complete mental model of what happens at every point — from the moment a user types on the page to the moment a result appears on screen.

1. User Authentication Flow

1.1 Registration

User fills signup form

- POST /api/v1/auth/register {username, email, password}
- accounts/views.py validates and creates Account record (is_active=False)
- Email sent with activation link containing a random token
- User clicks email link
- GET /api/v1/auth/activate/<token>
- account.is_active = True saved to MySQL
- User can now log in

1.2 Login

User enters username + password

- POST /api/v1/auth/login {username, password}
- accounts/views.py authenticates credentials
- DRF creates or returns an Auth Token record (stored in MySQL: authtoken_token table)
- Response: { token: "abc123..." }
- Frontend stores token in a cookie (via js-cookie)
- All subsequent API requests include: Authorization: Token abc123...

2. Starting a New Equational Proof

Step-by-step trace

User action: On the equational reasoning page, the user enters a proof name, fills in the LHS goal (e.g., `(length (append L M))`), fills in the RHS goal (e.g., `(+ (length L) (length M))`), enables some UDF definitions, and clicks "Start Proof".

```
[Browser]
EquationalReasoningNew.js collects form data:
{
  lhsPremise: "(length (append L M))",
  rhsPremise: "(+ (length L) (length M))",
```

```

definitions: [ {label:"length", def_type:"...", expression:"..."}, ... ],
generics: [ {label:"L", type:"list", restrictions:"never-null"}, ... ]
}

```

→ POST /api/v1/equational/set-current-proof

→ Authorization: Token <user-token>

[Django – equational_reasoning_api/views.py → set_current_proof()]

1. Extract user from auth token
2. Create EquationalProof database record with lhs_goal, rhs_goal, name, tag
3. Instantiate TwoSidedProof() in memory (ERProofEngine.py)
4. For each definition: call proof_obj.addUDF(label, type, body)
 - Parser.makeBasicAst(body) → Node tree
 - Registered in proof_obj.ruleSet as a UDF rule
5. For each generic: call proof_obj.addGeneric(label, type, restrictions)
 - Creates GenericInt/GenericList/GenericBool/GenericAny
 - Registered in proof_obj.generics dict
6. Parse LHS premise: Parser.makeBasicAst("(length (append L M))")
 - Returns Node tree rooted at "("
 - expressionDefinition.labelTree() assigns initial types
 - Decorator propagates types
 - makeJson(tree) converts to flat JSON dict keyed by startPosition
7. Create EquationalProofLine database record for LHS premise (line_number=0)
 - racket = "(length (append L M))"
 - json_tree = { "0": {...}, "1": {...}, ... }
 - rule = "" (no rule applied to premises)
8. Repeat steps 6-7 for RHS premise
9. Serialize TwoSidedProof object to bytes with dill.dumps()
 - Store in Django cache under key "proof_<user_token>"
10. Return JSON response:


```

{
  isValid: true,
  lhsPremise: "(length (append L M))",
  rhsPremise: "(+ (length L) (length M))",
  lhsJsonTree: { "0": {...}, ... },
  rhsJsonTree: { "0": {...}, ... }
}

```

[Browser]

- racketRuleFields state array is initialized with two entries:
 - [0]: { racket: "(length (append L M))", jsonTree: {...}, rule: "", ... } ← LHS premise
 - [1]: { racket: "", jsonTree: {}, rule: "", ... } ← blank trailing slot
- PersistentPad components render for each entry

3. Selecting a Node (Subexpression) in a Proof Line

User action: The student clicks on a subterm in a rendered proof line — for example, clicking on (append L M) inside (length (append L M)).

[Browser – PersistentPad.js]

- The expression is displayed as colored/highlighted text spans
- Each span corresponds to a range of character positions in the expression string
- User clicks a span
- onClick handler identifies the startPosition of the clicked span (e.g., startPosition = 9)
- PersistentPad sets internal state: selected = 9
- The span at position 9 is highlighted (e.g., yellow background)
- PersistentPad's ref exposes getStartPosition() → returns 9
- sessionStorage is updated to persist this selection:

```
{ key: "2-LHS-hash", equation: "...", selected: 9, ... }
```

Arrow key navigation (no API call needed):

- Arrow UP: selected = jsonTree[selected].parent
- Arrow DOWN: selected = jsonTree[selected].children[0]
- Arrow LEFT: selected = jsonTree[selected].leftSib
- Arrow RIGHT: selected = jsonTree[selected].rightSib

The `jsonTree` data structure, already in browser memory from the initial proof setup, enables this navigation. The parent, children, leftSib, rightSib values are all startPosition integers that act as node IDs.

4. Applying a Rule (Standard Rule Application)

User action: The student has selected node at startPosition=9 (the (append L M) subterm) and types the rule "eval append" in the rule input field, then clicks "GENERATE & CHECK".

[Browser – EquationalReasoningNew.js]

- Read selected node position from PersistentPad ref: startPosition = 9
- Read rule from input field: "eval append"
- Determine current side: "LHS"
- Current racket (last line on LHS): "(length (append L M))"
- Line number: 1 (zero-indexed after the premise)
- Call equationalService.applyRule({


```
side: "LHS",
currentRacket: "(length (append L M))",
rule: "eval append",
startPosition: 9,
selectedNode: 9,
lineNumber: 1
```

 })
 - POST /api/v1/equational/apply-rule
 - Authorization: Token <user-token>

[Django – equational_reasoning_api/views.py → apply_rule()]

1. Extract user from token; look up their EquationalProof record
2. Load serialized TwoSidedProof from cache:

```
dill.loads(cache.get("proof_<token>"))
```

3. If cache miss: rebuild proof from DB (iterate EquationalProofLine records, re-parse each expression and re-apply each rule)
4. Identify target side: proof_obj.LHS (an ERProof instance)
5. Load existing LHS proof lines from DB back into the ERProof:


```
for each EquationalProofLine(side='LHS') in order:
    target.proofLines.append(reconstructed ERProofLine)
```
6. Call target.addProofLine(


```
lineStr="(length (append L M))",
ruleStr="eval append",
highlightPos=9,
substitution=None
)
```

[Inside ERProofEngine.ERProof.addProofLine()]

 - a. Create a new ERProofLine for the result
 - b. ERProofLine parses "eval append": splits into ruleType="eval", label="append"
 - c. Looks up label in ruleSet: finds UDF rule for "append"
 - d. Calls Parser.makeBasicAst("(length (append L M))") → Node tree
 - e. expressionDefinition.labelTree(tree) assigns types to all nodes
 - f. Decorator validates types
 - g. Locates node at startPosition=9 using ERCommon.findNode()
 - h. Calls rule.isApplicable(node_at_9): verifies the node is a call to append
 - i. Calls rule.insertSubstitution(node_at_9):
 - Deep copies the append UDF body tree
 - Replaces parameter L with the first child of node_at_9
 - Replaces parameter M with the second child of node_at_9
 - Returns the substituted Node tree
 - j. Calls node_at_9.replaceWith(result_node): in-place tree mutation
 - k. Updates startPositions on all nodes: updatePositions(tree, count=0)
 - l. Converts result to string: "(length (if (null? L) M (cons (first L) (append (rest L) M))))"
 - m. Calls makeJson(result_tree) → flat JSON dict
 - n. Returns (result_racket_string, result_json, result_node_id, errors)
7. Back in view: save result to DB
 - a. Update previous EquationalProofLine (line_number=1): set selected_node=9
 - b. Create new EquationalProofLine (line_number=2):


```
racket = "(length (if (null? L) M ...))"
json_tree = { ... }
rule = "eval append"
start_position = 9
result_node = <id of replaced node in result>
errors = ""
```
8. Serialize updated TwoSidedProof back to cache
9. Return JSON:


```
{
  isValid: true,
  racket: "(length (if (null? L) M (cons (first L) (append (rest L) M))))",
  jsonTree: { ... },
  rule: "eval append",
  resultNodeId: <int>,
  errors: []
```

```
}
```

```
[Browser]
```

- Append new entry to racketRuleFields array (or update current entry)
- PersistentPad for new line renders the result expression
- Result node is highlighted (yellow) based on resultNodeId
- A new blank trailing slot is created

5. Applying a Substitution Rule

Substitution is a special rule application where the user must explicitly provide the expression to match before the rule is expanded. It is used when applying `eval` rules that require structural matching.

User action: The student wants to apply the inductive hypothesis at a specific place. They click "SUBST" to open the substitution modal, fill in the substitution expression, and submit.

```
[Browser – Substitution.jsx modal]
```

- User enters substitution expression: "(length K)"
- User selects rule: "rewrite IH"
- Calls `equationalService.substitution({`
 - `side: "LHS",`
 - `currentRacket: "(+ (length K) 1)",`
 - `rule: "rewrite IH",`
 - `startPosition: 3,`
 - `selectedNode: 3,`
 - `substitution: "(length K)",`
 - `lineNumber: 3`
- `})`
- POST /api/v1/equational/substitution

```
[Django – views.py → substitution()]
```

1. Same setup as `apply_rule` (load cache, load DB lines)
2. Call `target.addProofLine(`
 - `lineStr="(+ (length K) 1)",`
 - `ruleStr="rewrite IH",`
 - `highlightPos=3,`
 - `substitution="(length K)"`
- `)`

```
[Inside ERProofLine.applySubstitution()]
```

- a. Parse substitution string "(length K)" → secondary Node tree (subLine)
- b. Locate node at `startPosition=3` in current expression tree
- c. Call `isMatch(subLine.exprTree, node_at_3):`
 - Recursively compare tree structures
 - Returns True if structures match (modulo variable names)
- d. If match fails: return error "Substitution does not match selected node"
- e. If match succeeds: apply IH rule at `node_at_3`
 - Lookup "IH" rule in `ruleSet` (IH stores `indHypLHS` and `indHypRHS` strings)
 - Find which side of IH matches the selected node
 - Replace node with the other side of IH

f. Return result

3. Save result to DB; update cache
4. Return result JSON to frontend

6. Checking Proof Completion

User action: The student believes the proof is complete and clicks "Check Completion".

```
[Browser]
→ POST /api/v1/equational/check-completion

[Django - views.py → check_completion()]
1. Load TwoSidedProof from cache
2. Reload all LHS and RHS lines from DB
3. Call proof_obj.checkComplete()

    [Inside TwoSidedProof.checkComplete()]
    a. Walk LHS proofLines to find last non-blank line
      → last_lhs_expr = "(+ (length L) (length M))"
    b. Walk RHS proofLines to find last non-blank line
      → last_rhs_expr = "(+ (length L) (length M))"
    c. Compare string representations: last_lhs_expr == last_rhs_expr → True
    d. Verify no blank lines except the trailing slot → True
    e. Return True

4. Check all DB lines for hide_expression or hide_justification flags
   → If any hidden line exists: isComplete = False (instructor must remove hidden
   fields)
5. If all checks pass: update EquationalProof record: is_complete=True
6. Return: { isComplete: true, message: "Proof complete!" }
```

```
[Browser]
- isComplete = true → triggers ProofComplete.jsx confetti animation
- Completion flag also saved to racketRuleFields state
```

7. Saving and Loading a Proof

Saving

```
[Browser]
- User clicks "Save Proof"
  → POST /api/v1/equational/save-proof { proofId: 42, name: "My Proof" }
  → Django updates EquationalProof.name and sets is_active=True
  → Returns success
```

Loading

[Browser]

- User navigates to proof history and selects a saved proof
→ GET /api/v1/equational/get-user-proof?proofId=42

[Django]

1. Fetch EquationalProof record from DB
2. Fetch all EquationalProofLine records for this proof, ordered by line_number
3. Rebuild TwoSidedProof in memory by replaying all saved lines
4. Store rebuilt proof in cache
5. Return all line data as JSON:

```
{
  LHS: [
    { racket: "(length (append L M))", jsonTree: {...}, rule: "", ... },
    { racket: "(length ...)", jsonTree: {...}, rule: "eval append", ... },
    ...
  ],
  RHS: [
    { racket: "(+ (length L) (length M))", jsonTree: {...}, rule: "", ... },
    ...
  ]
}
```

[Browser]

- racketRuleFields re-initialized from response data
- All proof lines rendered by PersistentPad components
- Selections restored from sessionStorage where available

8. Induction Proof Data Flow

Induction proof data flow is similar to equational but adds the following dimensions:

1. **Case parameter:** Every API call includes `case: "base"` or `case: "leap"`. This determines which sub-proof (`proof_obj.baseCase` or `proof_obj.leapStep`) is targeted.
2. **Proof initialization** (`start_induction_proof`):
 - Takes additional parameters: `inductionType` (int/list), `ivar` (induction variable), `aval` (anchor value), `lvar` (leap variable).
 - Backend computes the LHS and RHS goals for both the base case and leap step by substituting `aval` and `lvar` into the original premise.
 - Creates the inductive hypothesis (`indHypLHS`, `indHypRHS`) by applying the substitution that replaces `ivar` with `lvar`.
 - Stores an `IH` rule in the leap step's rule set, carrying the hypothesis expressions.
3. **Completion check** (`check_goal`):
 - Checks each side (base/leap) separately.

- Proof fully complete only when both base case and leap step are complete.

9. Definition and Generic Setup

Before a proof, definitions and generics are sent from the client to configure the proof engine:

[Browser – Definitions.jsx]

- User enables/disables UDFs; adds custom definitions
- sessionStorage updated: definitions array persisted locally
- On "Start Proof": all enabled definitions + all generics from sessionStorage are included in the set-current-proof payload

[Django – set_current_proof()]

For each definition {label, def_type, expression}:

- proof_obj.addUDF(label, def_type, expression)
 - Parses expression to Node tree
 - Creates UDF rule instance
 - Adds to proof_obj.ruleSet[label]
 - Also adds to proof_obj.LHS.ruleSet and proof_obj.RHS.ruleSet

For each generic {label, type, restrictions}:

- proof_obj.addGeneric(label, type, restrictions)
 - Creates GenericInt/GenericList/GenericBool/GenericAny
 - Stores in proof_obj.generics[label]
 - When Parser encounters this label later, it assigns the generic's type

10. Error Propagation

Errors are generated in the proof engine and flow back to the frontend as follows:

[In expression_tree/]

- ERProofLine.errLog: list of error strings accumulated during parse + rule application
- Each error has a source (parser error, type error, rule-not-applicable, etc.)
- Errors are returned with the proof line result

[In views.py]

- errors extracted from proof line after addProofLine()
- Stored in EquationalProofLine.errors (comma-separated string in DB)
- Included in JSON response: errors: ["Error: rule not applicable at this position"]

[In Browser]

- equationalService.applyRule() receives errors array
- If errors.length > 0: displayed in the proof line row (red text below line)
- The expression in the line is NOT updated if a fatal error occurred
- User can try a different rule or node selection

11. Data Persistence Summary

Data type	Where stored	When written	When read
User accounts	MySQL (accounts_account)	On signup; on profile update	On every authenticated request
Auth tokens	MySQL (authtoken_token)	On login	On every API request
Proof metadata	MySQL (equationalproof / inductionproof)	On proof creation; on completion	On proof load
Proof lines	MySQL (equationalproofline / inductionproofline)	After each rule application	On proof load; on check_completion
Definitions	MySQL (proofs_definition)	When user saves a definition	When starting a new proof
Generics	MySQL (proofs_generic)	When user saves a generic	When starting a new proof
Active proof object	Django DB cache	After every rule application	At start of every API request
Node selections	Browser sessionStorage	On every node click	On proof load / component re-render
UDFs (session)	Browser sessionStorage	When user enables/changes a definition	When starting a proof